

IF2224: Teori Bahasa Formal dan Automata
Pembuatan Semantic Analyser
Untuk Bahasa Pemrograman Arion



Dipersiapkan oleh:

13524008	Agatha Tatianingseto
13524030	Irvin Tandiarrang Sumual
13524074	Bernhard Aprillio Pramana
13524096	Moreno Syawali Ganda Sugita
13524110	Jennifer Khang

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2025

Bab 1

Deskripsi Tugas

Melanjutkan tahapan kompilasi setelah *syntax analysis* selesai, sebuah program kompilator akan memasuki tahap *semantic analysis*. Jika *parser* bertugas memeriksa urutan token agar membentuk *Parse Tree* yang valid secara sintaksis, maka *semantic analysis* bertugas memeriksa makna dari program tersebut. Tahapan ini sangat esensial untuk memastikan bahwa struktur kode yang benar juga valid secara logika komputasional, seperti memastikan operator dan operand memiliki tipe data yang kompatibel melalui *type checking* serta memastikan setiap variabel telah dideklarasikan sebelum digunakan melalui *symbol table management*.

Pada tahap *semantic analysis* ini, program pada dasarnya akan mengonversi *Parse Tree* menjadi *Abstract Syntax Tree* (AST) dengan membuang informasi sintaksis yang sudah tidak lagi diperlukan untuk tahapan analisis selanjutnya, seperti tanda kurung berlebih atau kata kunci terminal. Untuk mengelola informasi semantik program, tahapan ini menggunakan tiga jenis tabel simbol, yaitu *tab* untuk menyimpan identifier, *btab* untuk informasi blok prosedur atau record, dan *atab* untuk informasi array. Evaluasi kebenaran semantik dan pengelolaan tabel ini dilakukan menggunakan pendekatan *Attributed Grammar*, di mana program menelusuri struktur pohon secara *top-down* memanfaatkan fungsi *visit* (*semantic visitor*) pada setiap simpulnya.

Dalam konteks tugas ini, Arion Compiler - Semantic Analysis merupakan pengembangan tahap ketiga dari kompilator bahasa pemrograman Arion. Program yang dibuat harus mampu menerima masukan berupa *Parse Tree* yang telah dihasilkan pada *milestone* sebelumnya. Melalui proses penelusuran, program akan menghasilkan keluaran berupa *Decorated AST* yang telah dianotasi dengan informasi tipe data dan referensi simbol, beserta struktur *Symbol Table* yang terkelola dengan baik. Selain itu, program juga dituntut untuk melakukan *scope resolution* dan mengevaluasi *type compatibility* pada seluruh instruksi kode. Keluaran AST yang telah terdekorasi ini sangat krusial karena akan mempermudah dan menjadi fondasi utama bagi proses kompilasi paling akhir nantinya.

Bab 2

Teori Singkat

2.1. *Semantic Analysis*

Semantic Analysis merupakan fase ketiga dalam proses kompilasi yang bertugas untuk memvalidasi makna atau semantik dari sebuah *source code* setelah program tersebut lolos dari tahapan analisis sintaksis (*parsing*). Meskipun sebuah program memiliki struktur tata bahasa yang benar (bebas dari *syntax error*), kompilator tetap harus memastikan bahwa operasi-operasi yang dilakukan pada program tersebut masuk akal secara komputasional dan mematuhi aturan bahasa pemrograman yang digunakan. Pengecekan pada tahapan ini mencakup *scope resolution*, *undeclared variables*, serta *type checking* pada ekspresi dan operasi *assignment*.

2.2. *Abstract Syntax Tree (AST) & Decorated AST*

Dalam merepresentasikan struktur hierarkis program, analisis semantik menggunakan *Abstract Syntax Tree (AST)* sebagai struktur data utamanya. Berbeda dengan *Parse Tree* yang menyimpan seluruh detail turunan sintaksis termasuk tanda baca dan *keywords*, AST menyederhanakan pohon tersebut dengan membuang token yang tidak lagi dibutuhkan untuk analisis logika, sehingga hanya merepresentasikan hubungan murni antara operator dan operannya. Proses kompilasi kemudian mengaplikasikan skema *Syntax-Directed Translation (SDT)* untuk mengevaluasi atribut dari setiap simpul pada AST secara *top-down* maupun *bottom-up*. Pohon sintaks abstrak yang telah dilengkapi dengan nilai atribut, seperti tipe data dan penunjuk lingkup, sering disebut sebagai *Decorated AST*.

2.3. *Symbol Table Management*

Symbol Table adalah sebuah struktur data penting yang dibuat dan di-maintain oleh kompilator guna menyimpan informasi mengenai berbagai identifier yang muncul di dalam *source code*, seperti nama variabel, konstanta, prosedur, tipe data bentukan, dan fungsi. Tabel ini bertindak sebagai basis data memori bagi kompilator untuk melacak properti krusial dari setiap identifier, meliputi tipe datanya, lokasi memori (*address*), hingga tingkatan *lexical scope* tempat identifier tersebut dideklarasikan. Dalam implementasi bahasa pemrograman yang kompleks, tabel simbol umumnya dipisahkan ke dalam

beberapa tabel spesifik, misalnya tabel khusus identifier umum, tabel blok *scope*, dan tabel struktur array untuk memfasilitasi *lookup* dan manajemen memori blok kode secara efisien dengan menggunakan konsep *stack*.

2.4. Aturan *Type Compatibility*

Type Checking adalah proses fundamental di dalam analisis semantik yang bertugas memverifikasi bahwa tipe data dari operan-operan yang terlibat dalam sebuah operator sesuai dengan operasi yang diizinkan. Aturan yang mendasari proses ini dikenal sebagai *Type Compatibility Rules*. Sistem ini harus memastikan kompatibilitas antar variabel yang sedang dioperasikan, contohnya mencegah *assignment* nilai bertipe *String* ke dalam variabel bertipe *Integer*. Selain itu, pengecekan ini juga memvalidasi batas nilai indeks (*bounds checking*) pada tipe data terstruktur seperti Array dan *Subrange* agar selaras dengan tipe indeks yang dideklarasikan secara konseptual.

2.5. *Scope Resolution*

Scope Resolution adalah tahapan analisis semantik yang menentukan validitas akses *identifier* (variabel, konstanta, atau subprogram) berdasarkan hierarki blok leksikal kode. Mekanisme ini memastikan setiap pemanggilan *identifier* merujuk pada deklarasi yang terikat secara logis, dengan membedakan *global scope* (dapat diakses dari seluruh bagian program) dan *local scope* (terbatas pada blok spesifik tempat deklarasi berada, seperti di dalam fungsi atau prosedur). Proses ini umumnya diimplementasikan menggunakan *Symbol Table* untuk melacak *lexical level* untuk memvalidasi visibilitas dan menangani penimpaan variabel (*shadowing*).

2.6. *Control Flow Validation*

Control Flow Validation adalah proses analisis untuk memastikan bahwa struktur kendali program memiliki alur eksekusi yang logis dan mematuhi kaidah semantik bahasa pemrograman. Pengecekan ini bertujuan untuk mencegah terjadinya anomali eksekusi atau kegagalan sistem. Contoh implementasi validasi ini meliputi:

- Memastikan instruksi lompatan seperti *break* atau *continue* secara eksklusif hanya berada di dalam blok *loop*.

- Memvalidasi bahwa setiap fungsi memiliki jaminan nilai kembalian (*return value*) pada seluruh kemungkinan cabang eksekusi kondisional.
- Mendeteksi dan memberikan peringatan terhadap keberadaan *unreachable code* (segmen kode yang tidak memiliki jalur eksekusi untuk dicapai).

Bab 3

Perancangan dan Implementasi

3.1. Arsitektur Utama Semantic Analyzer

Implementasi analisis semantik pada kompilator ini dibangun menggunakan kelas `SemanticAnalyzer` yang mengadopsi pola desain *Visitor Pattern* melalui antarmuka `ASTVisitor`. Pola ini dipilih karena kemampuannya memisahkan algoritma analisis dari struktur data *Abstract Syntax Tree* (AST) itu sendiri.

Proses analisis dimulai ketika *parser* menyerahkan *root node* bertipe `ASTProgramNode` kepada metode `analyze()`. Dari titik ini, *Semantic Analyzer* akan melakukan penelusuran (*traversal*) secara rekursif ke seluruh cabang pohon menggunakan metode `safeVisitNode()`. Setiap jenis *node* (misalnya `ASTAssignmentStatementNode` atau `ASTIfStatementNode`) ditangani oleh fungsi `visit` yang spesifik (*method overriding*). Pendekatan ini memastikan setiap instruksi dan ekspresi dievaluasi berdasarkan konteks leksikal dan tipe datanya, sambil secara dinamis memperbarui status variabel di dalam *Symbol Table*.

3.2. Implementasi *Symbol Table* dan *Scope*

Symbol Table pada kompilator ini dirancang secara modular dan dipecah menjadi tiga tabel utama untuk menangani berbagai kompleksitas tipe data:

1. *Identifier Table* (TAB)

Diimplementasikan melalui *struct* `IdentifierTableEntry`, tabel ini menyimpan informasi utama seluruh *identifier* seperti nama, tipe data dasar (`DataType`), level, dan offset alamat memori. Tabel ini juga diperkuat dengan *flag* `isConstant` untuk melacak mutabilitas dan vektor `parameterTypes` untuk memvalidasi *signature* fungsi.

2. *Array Table* (ATAB)

Diimplementasikan melalui *struct* `ArrayTableEntry`, tabel ini secara khusus menyimpan metadata tipe data *Array*, mencakup batas indeks (low dan high), tipe indeks, tipe elemen, dan ukuran total alokasi memori.

3. *Block Table* (BTAB)

Menyimpan metadata untuk setiap blok *scope* (fungsi, prosedur, atau *record*), mencakup ukuran parameter dan variabel lokal.

Scope Resolution diatur menggunakan metode `enterBlock()` dan `exitBlock()`. Saat penelusuran AST memasuki blok fungsi atau prosedur baru, `enterBlock()` dipanggil untuk menaikkan kedalaman leksikal (*current level*). Semua deklarasi di dalam blok ini akan diikat pada level tersebut. Saat penelusuran keluar dari blok, `exitBlock()` dieksekusi, sehingga *identifier* lokal yang terdaftar di dalam blok tersebut tidak dapat lagi diakses oleh blok di luar lingkupnya, mencegah terjadinya tabrakan nama dan menangani konsep *shadowing* dengan akurat.

3.3. Representasi dan Evaluasi Tipe Data

Analisis semantik pada kompilator Arion mengandalkan sistem representasi tipe data yang didefinisikan secara eksplisit melalui enumerasi `DataType`. Sistem ini dirancang untuk dapat menangani evaluasi tipe data primitif maupun tipe data komposit/bentukan hasil deklarasi pengguna secara dinamis.

3.3.1 Inisialisasi Pengenal Bawaan (Predefined Identifiers)

Pada saat objek `SemanticAnalyzer` pertama kali diinstansiasi, metode `initializePredefinedIdentifiers()` dijalankan secara otomatis untuk mendaftarkan elemen bawaan ke dalam *Symbol Table*. Proses inisialisasi ini mencakup:

- Tipe Data Dasar

Pengenal dasar seperti integer, real, char, boolean, dan string dimasukkan ke tabel untuk bertindak sebagai acuan validasi tipe bentukan selanjutnya.

- Konstanta Logika Bawaan

Pengenal `true` dan `false` didaftarkan dengan tipe data `DataType::BOOLEAN`. Untuk menjamin integritas logika program, kedua pengenal ini dikunci dengan menyetel atribut `isConstant = true` dan `normal = true`. Hal ini memastikan bahwa nilai logika bawaan terlindung dan tidak dapat ditulis ulang melalui instruksi di dalam *source code*.

- Subprogram Sistem

Prosedur cetak bawaan seperti `writeln` dan `println` didaftarkan ke dalam tabel dengan tipe data `DataType::VOID`.

3.3.2 Evaluasi Tipe Data Komposit dan Bentuk

Untuk tipe data bentuk yang lebih kompleks, SemanticAnalyzer melakukan evaluasi struktural menggunakan beberapa metode pembantu:

- Tipe Subrange (ASTRangeType)
Validasi dilakukan melalui fungsi `extractRangeBounds` untuk memastikan *lower bound* dan *upper bound* memiliki tipe ordinal yang sama (seperti sesama integer atau sesama char) serta memastikan batas bawah tidak bernilai lebih besar dari batas atas. Evaluasi ini juga secara ketat melarang penggunaan tipe data real sebagai komponen pengindeksan subrange.
- Tipe Array (ASTArrayTypeNode)
Manajemen struktur array diatur melalui fungsi `ensureArrayTypeEntry()`. Fungsi ini mengevaluasi komponen indeks array (memastikan tipenya ordinal dan bukan real) serta tipe elemennya. Struktur array kemudian didaftarkan ke dalam tabel array (`atab_`) dengan menghitung ukuran elemen (`elementSize`) dan total alokasi memori (`size`) secara presisi untuk memfasilitasi alokasi stack frame memori.
- Tipe Record (ASTRecordTypeNode)
Penanganan struktur record dilakukan dengan memeriksa setiap field di dalam blok definisi field. Fungsi pengecekan memastikan tidak ada nama pengenalan field yang duplikat dalam satu lingkup record yang sama.

3.4. Validasi Semantik dan Aturan Bahasa

Validasi semantik merupakan komponen utama kompilator untuk menjamin bahwa kode objek yang dihasilkan akan aman dari *runtime errors*. Terdapat tiga poin validasi utama yang diimplementasikan pada komponen ini:

3.4.1 Type Checking

Evaluasi kompatibilitas tipe data diimplementasikan melalui metode `validateAssignmentCompatibility()` dan `areTypesCompatible()`. Aturan tegas yang diterapkan oleh sistem meliputi:

- Assignment Primitif
Assignment diizinkan jika target dan ekspresi memiliki tipe data primitif yang identik (misalnya sesama integer atau sesama boolean). Pengecualian *type*

coercion secara otomatis hanya diberikan jika target bertipe REAL dan nilai ekspresi bertipe INTEGER.

- Assignment Komposit

Untuk struktur data komposit seperti *Record*, sistem memeriksa kesetaraan nama tipe data secara ketat melalui metode `areNamedRecordsCompatible()`. Dua objek record hanya kompatibel jika keduanya merujuk pada definisi tipe bernama yang sama, penggunaan *anonymous record* dengan struktur yang mirip tetap dianggap tidak kompatibel. Untuk *Array*, pengecekan kompatibilitas dilakukan secara rekursif pada tipe indeks dan tipe elemennya.

3.4.2 Pengecekan Subprogram (*Arity and Parameter Checking*)

Validasi pemanggilan fungsi dan prosedur diimplementasikan secara terpisah di dalam fungsi `visitCallExpressionNode()` (untuk fungsi di dalam ekspresi) dan `visitCallStatementNode()` (untuk prosedur sebagai statement mandiri). Validasi ini mencakup dua lapis pemeriksaan:

- Pemeriksaan Aritas (*Arity Checking*)

Kompilator membandingkan jumlah argumen yang dikirim pada AST (`arguments.size()`) dengan jumlah parameter yang tercatat dalam vektor `parameterTypes` di dalam *Symbol Table*. Jika jumlahnya tidak cocok, kompilator akan langsung menghentikan proses dan melempar *Semantic Error*.

- Pemeriksaan Tipe Parameter

Melalui iterasi terstruktur, tipe data dari setiap argumen hasil evaluasi (`argType`) dicocokkan secara posisional dengan tipe parameter yang diharapkan (`entry.parameterTypes[i]`). Ketidakcocokan tipe pada salah satu parameter akan memicu kegagalan semantik (*Type Mismatch*).

- Bypass Fungsi Bawaan

Khusus untuk fungsi bawaan sistem seperti `writeln` dan `println` yang bersifat *variadic* (menerima jumlah dan tipe argumen dinamis), sistem menerapkan mekanisme *bypass* pengondisian nama pengenalan agar argumennya cukup dievaluasi tanpa memicu kesalahan aritas tabel simbol.

3.4.3 Perlindungan Mutabilitas Konstanta

Perlindungan terhadap modifikasi data konstanta dilakukan pada level statement pengisian nilai di dalam metode `visitAssignmentStatementNode()`. Sebelum melakukan evaluasi tipe data, sistem terlebih dahulu melakukan pencarian pengenalan target di ruas kiri statement pada *Symbol Table*. Jika pengenalan tersebut ditemukan dan memiliki atribut `isConstant == true`, kompilator akan segera melempar *Semantic Error* yang menyatakan bahwa operasi pengisian nilai pada konstanta ilegal dilakukan. Mekanisme ini menjamin nilai konstanta *immutable* sepanjang eksekusi program berjalan.

3.5 Hasil Implementasi Validasi Semantik

Guna merealisasikan aturan-aturan semantik yang telah dirancang, dilakukan modifikasi dan ekstensi logika pada komponen Tabel Simbol serta modul *visitor* di dalam kompilator. Hasil dari implementasi ini memungkinkan kompilator untuk menangkap pelanggaran aturan bahasa dan menghentikan proses kompilasi dengan melemparkan eksepsi yang memuat lokasi (baris dan kolom). Berikut adalah cuplikan implementasi utama dari validasi semantik tersebut:

1. Ekstensi Meta-Data Tabel Simbol

```
Entries.hpp

bool normal;
int level;
int address;
std::string typeName;

// Ekstensi untuk validasi Semantik
bool isConstant;
std::vector<DataType> parameterTypes;
```

Struktur data `IdentifierTableEntry` diperluas dengan menambahkan atribut `isConstant` untuk melacak mutabilitas suatu pengenalan. Selain itu, ditambahkan pula struktur memori dinamis `parameterTypes` berbentuk vektor yang berfungsi untuk menyimpan urutan tipe data parameter pada saat sebuah fungsi atau prosedur dideklarasikan.

2. Implementasi Proteksi Konstanta

SemanticAnalyzer.hpp

```
    if (node->target->components.empty()) {
        int symbolIdx =
symbolTable.lookup(node->target->baseName);
        if (symbolIdx != -1 &&
symbolTable.getIdentifier(symbolIdx).isConstant) {
            throw std::runtime_error("Semantic Error: Tidak
dapat melakukan assignment pada konstanta '" +
node->target->baseName + "'.");
        }
    }
}
```

Sebelum melakukan evaluasi *assignment*, sistem akan meng-*query identifier* ruas kiri pada Tabel Simbol. Jika pengenal ditemukan dan memiliki penanda `isConstant == true` (seperti pada nilai literal bawaan `true/false` atau variabel konstan pengguna), kompilator membatalkan eksekusi *assignment* dan melempar *exception* untuk menjaga integritas konstanta.

3. Implementasi Pengecekan Aritas dan Tipe Argumen

SemanticAnalyzer.hpp

```
    IdentifierTableEntry& entry =
symbolTable.getIdentifier(symbolIdx);

    // Bypass untuk fungsi bawaan variadic
    if (node->callExpr->callee == "writeln" ||
node->callExpr->callee == "println") {
        for (auto* argument : node->callExpr->arguments) {
            safeVisitNode(argument);
        }
    } else {
        // Pengecekan Aritas (Jumlah Argumen)
        if (node->callExpr->arguments.size() !=
entry.parameterTypes.size()) {
            throw std::runtime_error("Semantic Error: Jumlah
argumen untuk prosedur '" + node->callExpr->callee + "' tidak
sesuai.");
        }
        // Pengecekan Kecocokan Tipe Data Argumen
        for (size_t i = 0; i <
node->callExpr->arguments.size(); ++i) {
            DataType argType =
std::any_cast<DataType>(node->callExpr->arguments[i]->accept(
this));
            if (argType != entry.parameterTypes[i]) {
                throw std::runtime_error("Semantic Error:
Type Mismatch pada parameter ke-" + std::to_string(i + 1) + "
```

```
    saat memanggil prosedur.");  
    }  
}
```

Implementasi ini bertugas memvalidasi pemanggilan subprogram. Langkah pertama adalah *Arity Checking*, yang membandingkan kesetaraan jumlah argumen aktual dengan jumlah parameter formal di memori. Langkah kedua adalah *Type Checking*, yang mengiterasi seluruh argumen untuk divalidasi kesesuaian tipenya secara sekuensial. Potongan kode ini juga mendemonstrasikan metode *bypass* khusus untuk menoleransi prosedur bawaan yang bersifat *variadic* (menerima argumen fleksibel) seperti `writeln`.

Bab 4

Pengujian

Pengujian 1

1. Nama Pengujian : Record
2. Input file : Record.txt
3. Output file : Record-ast.txt
4. Deskripsi : Pengujian pada AST dan Symbol Table pada program mengandung Record
5. Hasil Pengujian :

Tabel 4.1 Pengujian 1

Input	Output
<pre> program TestRecord; type Person == record age: integer; initial: char end; var p : Person; begin p.age := 20; p.initial := 'A'; writeln(p.age); writeln(p.initial); end.</pre>	<pre> Processing Semantic Analysis... Menganalisis Program: TestRecord Program: TestRecord {type=VOID, tab_index=37, lev=0} +--- Declarations {type=VOID, tab_index=-1, lev=0} +--- TypeDeclaration: Person {type=RECORD, tab_index=38, lev=0} \-- RecordType(age; initial) {type=RECORD, tab_index=-1, lev=0} +--- PrimitiveType: integer {type=INTEGER, tab_index=-1, lev=0} \-- PrimitiveType: char {type=CHAR, tab_index=-1, lev=0} \-- VarDecl(name: 'p', type: 'Person') {type=RECORD, tab_index=41, lev=0}</pre>

```

|
\-- Block {type=VOID, tab_index=-1, lev=0}
|
+-- Assignment {type=VOID, tab_index=-1, lev=0}
| |
| +-- Variable: p.age {type=INTEGER, tab_index=41, lev=0}
| |
| \-- Literal: 20 {type=INTEGER, tab_index=-1, lev=0}
|
+-- Assignment {type=VOID, tab_index=-1, lev=0}
| |
| +-- Variable: p.initial {type=CHAR, tab_index=41, lev=0}
| |
| \-- Literal: A {type=CHAR, tab_index=-1, lev=0}
|
+-- CallStatement {type=VOID, tab_index=-1, lev=0}
| |
| \-- Call: writeln {type=VOID, tab_index=33, lev=0}
| |
| \-- Variable: p.age {type=INTEGER, tab_index=41, lev=0}
|
+-- CallStatement {type=VOID, tab_index=-1, lev=0}
| |
| \-- Call: writeln {type=VOID, tab_index=33, lev=0}
| |
| \-- Variable: p.initial {type=CHAR, tab_index=41, lev=0}
|
\-- EmptyStatement {type=VOID, tab_index=-1, lev=0}

=== Symbol Tables ===

```

tab :

idx	id	obj	type	ref	nrm	lev	adr	link
-----	----	-----	------	-----	-----	-----	-----	------

0	<dummy>	keyword	void	0	1	0	0	0
1	and	keyword	void	0	1	0	4	0
2	array	keyword	void	0	1	0	8	1
3	begin	keyword	void	0	1	0	12	2
4	case	keyword	void	0	1	0	16	3
5	const	keyword	void	0	1	0	20	4
6	div	keyword	void	0	1	0	24	5
7	downto	keyword	void	0	1	0	28	6
8	do	keyword	void	0	1	0	32	7
9	else	keyword	void	0	1	0	36	8
10	end	keyword	void	0	1	0	40	9
11	for	keyword	void	0	1	0	44	10
12	function	keyword	void	0	1	0	48	11
13	if	keyword	void	0	1	0	52	12
14	mod	keyword	void	0	1	0	56	13
15	not	keyword	void	0	1	0	60	14
16	of	keyword	void	0	1	0	64	15
17	or	keyword	void	0	1	0	68	16
18	procedure	keyword	void	0	1	0	72	17
19	program	keyword	void	0	1	0	76	18
20	record	keyword	void	0	1	0	80	19
21	repeat	keyword	void	0	1	0	84	20
22	then	keyword	void	0	1	0	88	21
23	to	keyword	void	0	1	0	92	22
24	type	keyword	void	0	1	0	96	23
25	until	keyword	void	0	1	0	100	24
26	var	keyword	void	0	1	0	104	25

```

27 while    keyword void  0 1 0 108 26
28 integer  type    integer 0 1 0 112 27
29 real     type    real  0 1 0 116 28
30 char     type    char   0 1 0 120 29
31 boolean  type    boolean 0 1 0 124 30
32 string   type    string 0 1 0 128 31
33 writeln  procedure void  0 1 0 132 32
34 println  procedure void  0 1 0 136 33
35 true     constant boolean 0 1 0 140 34
36 false    constant boolean 0 1 0 144 35
37 TestRecord program void  0 1 0 148 36
38 Person   type    record 1 1 0 152 37
39 age      field   integer 0 1 1 0 0
40 initial  field   char   0 1 1 4 39
41 p        variabel record 0 1 0 157 38

```

btab :

idx last lparam psize vsize

```

-----
0 41 0 0 0
1 40 0 0 5

```

atab :

idx inxtyp eltyp ref low high elsz size

```

-----

Semantic report berhasil disimpan di:
../test/milestone-3/output/Record-ast.txt

```

Selesai.

Pengujian 2

1. Nama Pengujian : Case
2. Input file : Case.txt
3. Output file : Case-ast.txt
4. Deskripsi : Pengujian pada AST dan Symbol Table pada program mengandung Case
5. Hasil Pengujian :

Tabel 4.2 Pengujian 2

Input	Output
<pre> program TestCase; var x : integer; begin x := 2; case x of 1: writeln('one'); 2: writeln('two'); end; end. </pre>	<pre> Processing Semantic Analysis... Menganalisis Program: TestCase Program: TestCase {type=VOID, tab_index=37, lev=0} +-- Declarations {type=VOID, tab_index=-1, lev=0} \-- VarDecl(name: 'x', type: 'integer') {type=INTEGER, tab_index=38, lev=0} \-- Block {type=VOID, tab_index=-1, lev=0} +-- Assignment {type=VOID, tab_index=-1, lev=0} +-- Variable: x {type=INTEGER, tab_index=38, lev=0} \-- Literal: 2 {type=INTEGER, tab_index=-1, lev=0} +-- Case {type=VOID, tab_index=-1, lev=0} +-- Variable: x {type=INTEGER, tab_index=38, lev=0} +-- Literal: 1 {type=INTEGER, tab_index=-1, lev=0} </pre>

```

| |
| +-- CallStatement {type=VOID, tab_index=-1, lev=0}
| | |
| | \-- Call: writeln {type=VOID, tab_index=33, lev=0}
| | |
| | \-- Literal: one {type=STRING, tab_index=-1, lev=0}
| |
| +-- Literal: 2 {type=INTEGER, tab_index=-1, lev=0}
| |
| \-- CallStatement {type=VOID, tab_index=-1, lev=0}
| |
| \-- Call: writeln {type=VOID, tab_index=33, lev=0}
| |
| \-- Literal: two {type=STRING, tab_index=-1, lev=0}
|
\-- EmptyStatement {type=VOID, tab_index=-1, lev=0}

```

=== Symbol Tables ===

tab :

idx	id	obj	type	ref	nrm	lev	adr	link
-----	----	-----	------	-----	-----	-----	-----	------

0	<dummy>	keyword	void	0	1	0	0	0
1	and	keyword	void	0	1	0	4	0
2	array	keyword	void	0	1	0	8	1
3	begin	keyword	void	0	1	0	12	2
4	case	keyword	void	0	1	0	16	3
5	const	keyword	void	0	1	0	20	4
6	div	keyword	void	0	1	0	24	5
7	downto	keyword	void	0	1	0	28	6
8	do	keyword	void	0	1	0	32	7

9	else	keyword	void	0	1	0	36	8
10	end	keyword	void	0	1	0	40	9
11	for	keyword	void	0	1	0	44	10
12	function	keyword	void	0	1	0	48	11
13	if	keyword	void	0	1	0	52	12
14	mod	keyword	void	0	1	0	56	13
15	not	keyword	void	0	1	0	60	14
16	of	keyword	void	0	1	0	64	15
17	or	keyword	void	0	1	0	68	16
18	procedure	keyword	void	0	1	0	72	17
19	program	keyword	void	0	1	0	76	18
20	record	keyword	void	0	1	0	80	19
21	repeat	keyword	void	0	1	0	84	20
22	then	keyword	void	0	1	0	88	21
23	to	keyword	void	0	1	0	92	22
24	type	keyword	void	0	1	0	96	23
25	until	keyword	void	0	1	0	100	24
26	var	keyword	void	0	1	0	104	25
27	while	keyword	void	0	1	0	108	26
28	integer	type	integer	0	1	0	112	27
29	real	type	real	0	1	0	116	28
30	char	type	char	0	1	0	120	29
31	boolean	type	boolean	0	1	0	124	30
32	string	type	string	0	1	0	128	31
33	writeln	procedure	void	0	1	0	132	32
34	println	procedure	void	0	1	0	136	33
35	true	constant	boolean	0	1	0	140	34
36	false	constant	boolean	0	1	0	144	35
37	TestCase	program	void	0	1	0	148	36
38	x	variabel	integer	0	1	0	152	37

btab :

idx last lparam psize vsize

0 38 0 0 0

atab :

idx inxtyp eltyp ref low high elsz size

Semantic report berhasil disimpan di:

../test/milestone-3/output/Case-ast.txt

Pengujian 3

1. Nama Pengujian : IfElse
2. Input file : IfElse.txt
3. Output file : IfElse-ast.txt
4. Deskripsi : Pengujian pada AST dan Symbol Table pada program mengandung If dan Else
5. Hasil Pengujian :

Tabel 4.3 Pengujian 3

Input	Output
<pre> program ifelse; var age: integer; begin age := 25; if age >= 18 then writeln('Adult') else writeln('Minor'); writeln('Done') end. </pre>	<pre> Processing Semantic Analysis... Menganalisis Program: ifelse Program: ifelse {type=VOID, tab_index=37, lev=0} +-- Declarations {type=VOID, tab_index=-1, lev=0} \-- VarDecl(name: 'age', type: 'integer') {type=INTEGER, tab_index=38, lev=0} \-- Block {type=VOID, tab_index=-1, lev=0} +-- Assignment {type=VOID, tab_index=-1, lev=0} +-- Variable: age {type=INTEGER, tab_index=38, lev=0} \-- Literal: 25 {type=INTEGER, tab_index=-1, lev=0} +-- If {type=VOID, tab_index=-1, lev=0} +-- BinaryOp: >= {type=BOOLEAN, tab_index=-1, lev=0} </pre>

```

| | +-- Variable: age {type=INTEGER, tab_index=38, lev=0}
| | |
| | \-- Literal: 18 {type=INTEGER, tab_index=-1, lev=0}
| |
| +-- CallStatement {type=VOID, tab_index=-1, lev=0}
| | |
| | \-- Call: writeln {type=VOID, tab_index=33, lev=0}
| | |
| | \-- Literal: Adult {type=STRING, tab_index=-1, lev=0}
| |
| \-- CallStatement {type=VOID, tab_index=-1, lev=0}
| | |
| | \-- Call: writeln {type=VOID, tab_index=33, lev=0}
| | |
| | \-- Literal: Minor {type=STRING, tab_index=-1, lev=0}
|
\-- CallStatement {type=VOID, tab_index=-1, lev=0}
|
\-- Call: writeln {type=VOID, tab_index=33, lev=0}
|
\-- Literal: Done {type=STRING, tab_index=-1, lev=0}

```

=== Symbol Tables ===

tab :

idx	id	obj	type	ref	nrm	lev	adr	link
-----	----	-----	------	-----	-----	-----	-----	------

0	<dummy>	keyword	void	0	1	0	0	0
---	---------	---------	------	---	---	---	---	---

1	and	keyword	void	0	1	0	4	0
---	-----	---------	------	---	---	---	---	---

2	array	keyword	void	0	1	0	8	1
---	-------	---------	------	---	---	---	---	---

3	begin	keyword	void	0	1	0	12	2
---	-------	---------	------	---	---	---	----	---

4	case	keyword	void	0	1	0	16	3
5	const	keyword	void	0	1	0	20	4
6	div	keyword	void	0	1	0	24	5
7	downto	keyword	void	0	1	0	28	6
8	do	keyword	void	0	1	0	32	7
9	else	keyword	void	0	1	0	36	8
10	end	keyword	void	0	1	0	40	9
11	for	keyword	void	0	1	0	44	10
12	function	keyword	void	0	1	0	48	11
13	if	keyword	void	0	1	0	52	12
14	mod	keyword	void	0	1	0	56	13
15	not	keyword	void	0	1	0	60	14
16	of	keyword	void	0	1	0	64	15
17	or	keyword	void	0	1	0	68	16
18	procedure	keyword	void	0	1	0	72	17
19	program	keyword	void	0	1	0	76	18
20	record	keyword	void	0	1	0	80	19
21	repeat	keyword	void	0	1	0	84	20
22	then	keyword	void	0	1	0	88	21
23	to	keyword	void	0	1	0	92	22
24	type	keyword	void	0	1	0	96	23
25	until	keyword	void	0	1	0	100	24
26	var	keyword	void	0	1	0	104	25
27	while	keyword	void	0	1	0	108	26
28	integer	type	integer	0	1	0	112	27
29	real	type	real	0	1	0	116	28
30	char	type	char	0	1	0	120	29
31	boolean	type	boolean	0	1	0	124	30
32	string	type	string	0	1	0	128	31
33	writeln	procedure	void	0	1	0	132	32

```
34 println procedure void 0 1 0 136 33
35 true constant boolean 0 1 0 140 34
36 false constant boolean 0 1 0 144 35
37 ifelse program void 0 1 0 148 36
38 age variabel integer 0 1 0 152 37
```

btab :

idx last lparam psize vsize

0 38 0 0 0

atab :

idx inxtyp eltyp ref low high elsz size

Semantic report berhasil disimpan di:

../test/milestone-3/output/ifElse-ast.txt

Selesai.

Pengujian 4

1. Nama Pengujian : FunctionCall
2. Input file : FunctionCall.txt
3. Output file : FunctionCall-ast.txt
4. Deskripsi : Pengujian pada AST dan Symbol Table pada program mengandung FunctionCall
5. Hasil Pengujian :

Tabel 4.4 Pengujian 4

Input	Output
<pre> program TestFunctionCall; function Add(a : integer; b : integer): integer; begin Add := a + b; end; begin writeln(Add(2, 3)); end. </pre>	<pre> Processing Semantic Analysis... Menganalisis Program: TestFunctionCall Program: TestFunctionCall {type=VOID, tab_index=37, lev=0} +-- Declarations {type=VOID, tab_index=-1, lev=0} \-- FunctionDeclaration: Add(a; b) : integer {type=INTEGER, tab_index=38, lev=0} +-- PrimitiveType: integer {type=INTEGER, tab_index=-1, lev=0} +-- PrimitiveType: integer {type=INTEGER, tab_index=-1, lev=0} \-- Block {type=VOID, tab_index=-1, lev=0} +-- Assignment {type=VOID, tab_index=-1, lev=0} +-- Variable: Add {type=INTEGER, tab_index=38, lev=0} </pre>

```

|      | |
|      | \-- BinaryOp: + {type=INTEGER, tab_index=-1,
lev=0}
|      | |
|      | +-- Variable: a {type=INTEGER, tab_index=39,
lev=1}
|      | |
|      | \-- Variable: b {type=INTEGER, tab_index=40,
lev=1}
|      |
|      | \-- EmptyStatement {type=VOID, tab_index=-1, lev=0}
|
\-- Block {type=VOID, tab_index=-1, lev=0}
|
| +-- CallStatement {type=VOID, tab_index=-1, lev=0}
| |
| | \-- Call: writeln {type=VOID, tab_index=33, lev=0}
| |
| | \-- Call: Add {type=INTEGER, tab_index=38, lev=0}
| |
| | +-- Literal: 2 {type=INTEGER, tab_index=-1, lev=0}
| |
| | \-- Literal: 3 {type=INTEGER, tab_index=-1, lev=0}
|
\-- EmptyStatement {type=VOID, tab_index=-1, lev=0}

```

=== Symbol Tables ===

tab :

idx	id	obj	type	ref	nrm	lev	adr	link
-----	----	-----	------	-----	-----	-----	-----	------

0	<dummy>	keyword	void	0	1	0	0	0
1	and	keyword	void	0	1	0	4	0
2	array	keyword	void	0	1	0	8	1
3	begin	keyword	void	0	1	0	12	2
4	case	keyword	void	0	1	0	16	3
5	const	keyword	void	0	1	0	20	4
6	div	keyword	void	0	1	0	24	5
7	downto	keyword	void	0	1	0	28	6
8	do	keyword	void	0	1	0	32	7
9	else	keyword	void	0	1	0	36	8
10	end	keyword	void	0	1	0	40	9
11	for	keyword	void	0	1	0	44	10
12	function	keyword	void	0	1	0	48	11
13	if	keyword	void	0	1	0	52	12
14	mod	keyword	void	0	1	0	56	13
15	not	keyword	void	0	1	0	60	14
16	of	keyword	void	0	1	0	64	15
17	or	keyword	void	0	1	0	68	16
18	procedure	keyword	void	0	1	0	72	17
19	program	keyword	void	0	1	0	76	18
20	record	keyword	void	0	1	0	80	19
21	repeat	keyword	void	0	1	0	84	20
22	then	keyword	void	0	1	0	88	21
23	to	keyword	void	0	1	0	92	22
24	type	keyword	void	0	1	0	96	23
25	until	keyword	void	0	1	0	100	24
26	var	keyword	void	0	1	0	104	25
27	while	keyword	void	0	1	0	108	26
28	integer	type	integer	0	1	0	112	27
29	real	type	real	0	1	0	116	28

```

30 char      type   char  0  1  0 120 29
31 boolean   type   boolean 0  1  0 124 30
32 string    type   string 0  1  0 128 31
33 writeln   procedure void  0  1  0 132 32
34 println   procedure void  0  1  0 136 33
35 true      constant boolean 0  1  0 140 34
36 false     constant boolean 0  1  0 144 35
37 TestFunctionCall program void  0  1  0 148 36
38 Add       function integer 1  1  0 152 37
39 a         parameter integer 0  1  1  0  0
40 b         parameter integer 0  1  1  4  39

```

btab :

idx last lparam psize vsize

```

-----
0 38 0  0  0
1 40 40  8  8

```

atab :

idx inxtyp eltyp ref low high elsz size

Semantic report berhasil disimpan di:

../test/milestone-3/output/FunctionCall-ast.txt

=== Semantic Warnings ===

[line 6, col 1] Warning: Statement ini tidak akan pernah dijalankan karena statement sebelumnya sudah mengembalikan nilai function.

=====

	Selesai.
--	----------

Pengujian 5

1. Nama Pengujian : NestedArray
2. Input file : NestedArray.txt
3. Output file : NestedArray-ast.txt
4. Deskripsi : Pengujian pada AST dan Symbol Table pada program mengandung NestedArray atau Array dengan tipe elemen adalah sebuah Record
5. Hasil Pengujian :

Tabel 4.5 Pengujian 5

Input	Output
<pre> program NestedArray; type Person == record age : integer end; People == array[1..5] of Person; var p : People; begin p[1].age := 20; end. </pre>	<pre> Processing Semantic Analysis... Menganalisis Program: NestedArray Program: NestedArray {type=VOID, tab_index=37, lev=0} +-- Declarations {type=VOID, tab_index=-1, lev=0} +-- TypeDeclaration: Person {type=RECORD, tab_index=38, lev=0} \-- RecordType(age) {type=RECORD, tab_index=-1, lev=0} \-- PrimitiveType: integer {type=INTEGER, tab_index=-1, lev=0} +-- TypeDeclaration: People {type=ARRAY, tab_index=40, lev=0} \-- ArrayType {type=ARRAY, tab_index=-1, lev=0} +-- RangeType {type=VOID, tab_index=-1, lev=0} +-- Literal: 1 {type=VOID, tab_index=-1, lev=0} </pre>

```

| | | |
| | | \-- Literal: 5 {type=VOID, tab_index=-1, lev=0}
| | |
| | \-- NamedType: Person {type=VOID, tab_index=-1, lev=0}
| |
| \-- VarDecl(name: 'p', type: 'People') {type=ARRAY,
tab_index=41, lev=0}
|
\-- Block {type=VOID, tab_index=-1, lev=0}
|
+-- Assignment {type=VOID, tab_index=-1, lev=0}
| |
| +-- Variable: p[].age {type=INTEGER, tab_index=41, lev=0}
| | |
| | \-- Literal: 1 {type=INTEGER, tab_index=-1, lev=0}
| |
| \-- Literal: 20 {type=INTEGER, tab_index=-1, lev=0}
|
\-- EmptyStatement {type=VOID, tab_index=-1, lev=0}

```

=== Symbol Tables ===

tab :

idx	id	obj	type	ref	nrm	lev	adr	link
-----	----	-----	------	-----	-----	-----	-----	------

0	<dummy>	keyword	void	0	1	0	0	0
1	and	keyword	void	0	1	0	4	0
2	array	keyword	void	0	1	0	8	1
3	begin	keyword	void	0	1	0	12	2
4	case	keyword	void	0	1	0	16	3
5	const	keyword	void	0	1	0	20	4

6	div	keyword	void	0	1	0	24	5
7	downto	keyword	void	0	1	0	28	6
8	do	keyword	void	0	1	0	32	7
9	else	keyword	void	0	1	0	36	8
10	end	keyword	void	0	1	0	40	9
11	for	keyword	void	0	1	0	44	10
12	function	keyword	void	0	1	0	48	11
13	if	keyword	void	0	1	0	52	12
14	mod	keyword	void	0	1	0	56	13
15	not	keyword	void	0	1	0	60	14
16	of	keyword	void	0	1	0	64	15
17	or	keyword	void	0	1	0	68	16
18	procedure	keyword	void	0	1	0	72	17
19	program	keyword	void	0	1	0	76	18
20	record	keyword	void	0	1	0	80	19
21	repeat	keyword	void	0	1	0	84	20
22	then	keyword	void	0	1	0	88	21
23	to	keyword	void	0	1	0	92	22
24	type	keyword	void	0	1	0	96	23
25	until	keyword	void	0	1	0	100	24
26	var	keyword	void	0	1	0	104	25
27	while	keyword	void	0	1	0	108	26
28	integer	type	integer	0	1	0	112	27
29	real	type	real	0	1	0	116	28
30	char	type	char	0	1	0	120	29
31	boolean	type	boolean	0	1	0	124	30
32	string	type	string	0	1	0	128	31
33	writeln	procedure	void	0	1	0	132	32
34	println	procedure	void	0	1	0	136	33
35	true	constant	boolean	0	1	0	140	34


```
36 false    constant boolean 0  1  0 144 35
37 NestedArray program void  0  1  0 148 36
38 Person    type    record 1  1  0 152 37
39 age       field   integer 0  1  1  0  0
40 People    type    array  0  1  0 156 38
41 p         variabel array  0  1  0 176 40
```

btab :

idx last lparam psize vsize

```
-----
0 41 0  0  0
1 39 0  0  4
```

atab :

idx inxtyp eltyp ref low high elsz size

```
-----
0 range record 38 1  5  4  20
```

Semantic report berhasil disimpan di:

../test/milestone-3/output/NestedArray-ast.txt

Selesai.

Pengujian 6

1. Nama Pengujian : RecursiveFactorial
2. Input file : RecursiveFactorial
3. Output file : RecursiveFactorial-ast.txt
4. Deskripsi : Pengujian pada AST dan Symbol Table pada program mengandung Function Rekursif
5. Hasil Pengujian :

Tabel 4.6 Pengujian 6

Input	Output
<pre> program RecursiveFactorial; function Fact(n: integer): integer; begin if n == 0 then Fact := 1 else Fact := n * Fact(n - 1); end; begin writeln(Fact(5)); end. </pre>	<pre> Processing Semantic Analysis... Menganalisis Program: RecursiveFactorial Program: RecursiveFactorial {type=VOID, tab_index=37, lev=0} +-- Declarations {type=VOID, tab_index=-1, lev=0} \-- FunctionDeclaration: Fact(n) : integer {type=INTEGER, tab_index=38, lev=0} +-- PrimitiveType: integer {type=INTEGER, tab_index=-1, lev=0} \-- Block {type=VOID, tab_index=-1, lev=0} +-- If {type=VOID, tab_index=-1, lev=1} +-- BinaryOp: == {type=BOOLEAN, tab_index=-1, lev=0} +-- Variable: n {type=INTEGER, tab_index=39, lev=1} \-- Literal: 0 {type=INTEGER, tab_index=-1, lev=0} </pre>

		+++ Assignment {type=VOID, tab_index=-1, lev=0}
		+++ Variable: Fact {type=INTEGER, tab_index=38,
		lev=0}
		\-- Literal: 1 {type=INTEGER, tab_index=-1, lev=0}
		\-- Assignment {type=VOID, tab_index=-1, lev=0}
		+++ Variable: Fact {type=INTEGER, tab_index=38,
		lev=0}
		\-- BinaryOp: * {type=INTEGER, tab_index=-1, lev=0}
		+++ Variable: n {type=INTEGER, tab_index=39,
		lev=1}
		\-- Call: Fact {type=INTEGER, tab_index=38, lev=0}
		\-- BinaryOp: - {type=INTEGER, tab_index=-1,
		lev=0}
		+++ Variable: n {type=INTEGER, tab_index=39,
		lev=1}
		\-- Literal: 1 {type=INTEGER, tab_index=-1,
		lev=0}
		\-- EmptyStatement {type=VOID, tab_index=-1, lev=0}

```

|
|-- Block {type=VOID, tab_index=-1, lev=0}
|
|--- CallStatement {type=VOID, tab_index=-1, lev=0}
| |
| | |-- Call: writeln {type=VOID, tab_index=33, lev=0}
| | |
| | | |-- Call: Fact {type=INTEGER, tab_index=38, lev=0}
| | | |
| | | | |-- Literal: 5 {type=INTEGER, tab_index=-1, lev=0}
| | | |
| | | |-- EmptyStatement {type=VOID, tab_index=-1, lev=0}

```

=== Symbol Tables ===

tab :

idx	id	obj	type	ref	nrm	lev	adr	link

0	<dummy>		keyword	void	0	1	0	0
1	and		keyword	void	0	1	0	4
2	array		keyword	void	0	1	0	8
3	begin		keyword	void	0	1	0	12
4	case		keyword	void	0	1	0	16
5	const		keyword	void	0	1	0	20
6	div		keyword	void	0	1	0	24
7	downto		keyword	void	0	1	0	28
8	do		keyword	void	0	1	0	32
9	else		keyword	void	0	1	0	36
10	end		keyword	void	0	1	0	40
11	for		keyword	void	0	1	0	44
12	function		keyword	void	0	1	0	48

13	if	keyword	void	0	1	0	52	12
14	mod	keyword	void	0	1	0	56	13
15	not	keyword	void	0	1	0	60	14
16	of	keyword	void	0	1	0	64	15
17	or	keyword	void	0	1	0	68	16
18	procedure	keyword	void	0	1	0	72	17
19	program	keyword	void	0	1	0	76	18
20	record	keyword	void	0	1	0	80	19
21	repeat	keyword	void	0	1	0	84	20
22	then	keyword	void	0	1	0	88	21
23	to	keyword	void	0	1	0	92	22
24	type	keyword	void	0	1	0	96	23
25	until	keyword	void	0	1	0	100	24
26	var	keyword	void	0	1	0	104	25
27	while	keyword	void	0	1	0	108	26
28	integer	type	integer	0	1	0	112	27
29	real	type	real	0	1	0	116	28
30	char	type	char	0	1	0	120	29
31	boolean	type	boolean	0	1	0	124	30
32	string	type	string	0	1	0	128	31
33	writeln	procedure	void	0	1	0	132	32
34	println	procedure	void	0	1	0	136	33
35	true	constant	boolean	0	1	0	140	34
36	false	constant	boolean	0	1	0	144	35
37	RecursiveFactorial	program	void	0	1	0	148	36
38	Fact	function	integer	1	1	0	152	37
39	n	parameter	integer	0	1	1	0	0

btab :

idx last lparam psize vsize

```
-----  
0 38 0 0 0  
1 39 39 4 4
```

atab :

```
idx inxtyp eltyp ref low high elsz size  
-----
```

Semantic report berhasil disimpan di:

../test/milestone-3/output/RecursiveFactorial-ast.txt

=== Semantic Warnings ===

[line 9, col 1] Warning: Statement ini tidak akan pernah dijalankan
karena statement sebelumnya sudah mengembalikan nilai function.

=====

Selesai.

Pengujian 7

1. Nama Pengujian : Enum
2. Input file : Enum.txt
3. Output file : Enum-ast.txt
4. Deskripsi : Pengujian pada AST dan Symbol Table pada program mengandung Enumerator
5. Hasil Pengujian :

Tabel 4.7 Pengujian 7

Input	Output
<pre> program TestTypes; type Color == (Red, Green, Blue); var c: Color; begin writeln(c); end.</pre>	<pre> Processing Semantic Analysis... Menganalisis Program: TestTypes Program: TestTypes {type=VOID, tab_index=37, lev=0} +-- Declarations {type=VOID, tab_index=-1, lev=0} +-- TypeDeclaration: Color {type=ENUMERATED, tab_index=41, lev=0} \-- EnumeratedType {type=ENUMERATED, tab_index=-1, lev=0} \-- VarDecl(name: 'c', type: 'Color') {type=ENUMERATED, tab_index=42, lev=0} \-- Block {type=VOID, tab_index=-1, lev=0} +-- CallStatement {type=VOID, tab_index=-1, lev=0} \-- Call: writeln {type=VOID, tab_index=33, lev=0} </pre>

| \-- Variable: c {type=ENUMERATED, tab_index=42, lev=0}

|

\-- EmptyStatement {type=VOID, tab_index=-1, lev=0}

=== Symbol Tables ===

tab :

idx	id	obj	type	ref	nrm	lev	adr	link
-----	----	-----	------	-----	-----	-----	-----	------

0	<dummy>	keyword	void	0	1	0	0	0
1	and	keyword	void	0	1	0	4	0
2	array	keyword	void	0	1	0	8	1
3	begin	keyword	void	0	1	0	12	2
4	case	keyword	void	0	1	0	16	3
5	const	keyword	void	0	1	0	20	4
6	div	keyword	void	0	1	0	24	5
7	downto	keyword	void	0	1	0	28	6
8	do	keyword	void	0	1	0	32	7
9	else	keyword	void	0	1	0	36	8
10	end	keyword	void	0	1	0	40	9
11	for	keyword	void	0	1	0	44	10
12	function	keyword	void	0	1	0	48	11
13	if	keyword	void	0	1	0	52	12
14	mod	keyword	void	0	1	0	56	13
15	not	keyword	void	0	1	0	60	14
16	of	keyword	void	0	1	0	64	15
17	or	keyword	void	0	1	0	68	16
18	procedure	keyword	void	0	1	0	72	17
19	program	keyword	void	0	1	0	76	18
20	record	keyword	void	0	1	0	80	19
21	repeat	keyword	void	0	1	0	84	20

22	then	keyword	void	0	1	0	88	21
23	to	keyword	void	0	1	0	92	22
24	type	keyword	void	0	1	0	96	23
25	until	keyword	void	0	1	0	100	24
26	var	keyword	void	0	1	0	104	25
27	while	keyword	void	0	1	0	108	26
28	integer	type	integer	0	1	0	112	27
29	real	type	real	0	1	0	116	28
30	char	type	char	0	1	0	120	29
31	boolean	type	boolean	0	1	0	124	30
32	string	type	string	0	1	0	128	31
33	writeln	procedure	void	0	1	0	132	32
34	println	procedure	void	0	1	0	136	33
35	true	constant	boolean	0	1	0	140	34
36	false	constant	boolean	0	1	0	144	35
37	TestTypes	program	void	0	1	0	148	36
38	Red	constant	enumerated	0	1	0	0	37
39	Green	constant	enumerated	0	1	0	1	38
40	Blue	constant	enumerated	0	1	0	2	39
41	Color	type	enumerated	0	1	0	164	40
42	c	variabel	enumerated	0	1	0	168	41

btab :

idx last lparam psize vsize

0 42 0 0 0

atab :

idx inxtyp eltyp ref low high elsz size

	Semantic report berhasil disimpan di: ../test/milestone-3/output/Enum-ast.txt Selesai.
--	---

Pengujian 8

1. Nama Pengujian :Btab Atab
2. Input file : TestIsiBtabAtab.txt
3. Output file : TestIsiBtabAtab-ast.txt
4. Deskripsi : Pengujian pada pengisian symbol table, terutama Btab dan juga Atab
5. Hasil Pengujian :

Tabel 4.8 Pengujian 8

Input
<pre> program ArionTableFillerTest; type { * Mendefinisikan tipe array 1 dimensi untuk memicu registrasi di atab * } BarisData == array[1..3] of integer; { * Mendefinisikan Array dari Array (Multi-dimensi) * } { * Ini akan memaksa kolom 'ref' di atab terisi dengan indeks dari tipe 'BarisData' * } MatriksData == array[1..4] of BarisData; var myMatrix : MatriksData; globalCounter : integer; { * ----- * } { * Subprogram dengan banyak parameter untuk mengisi lparam & psize * } { * ----- * } procedure HitungKalkulasiKompleks(p1 : integer; p2 : integer; opsi : boolean; flag : char); var lokalX : integer; begin lokalX := p1 + p2; if opsi then writeln(flag) </pre>

```

    else
        writeln('Selesai');
end;

function PengecekanMatriks(m : MatriksData; batas : integer) : boolean;
begin
    if m[1][1] > batas then
        PengecekanMatriks := true
    else
        PengecekanMatriks := false;
end;

{
=====
*}
{* MAIN PROGRAM BODY                                *}
{*
=====
*}
begin
    globalCounter := 10;

    {* Mengisi data matriks *}
    myMatrix[1][1] := 5;

    {* Memanggil prosedur dengan parameter lengkap *}
    HitungKalkulasiKompleks(5, 10, true, 'A');

    if PengecekanMatriks(myMatrix, globalCounter) then
        writeln('Nilai matriks aman.')
    else
        writeln('Nilai matriks di bawah batas.');
```

Output

```

Processing Semantic Analysis...
Menganalisis Program: ArionTableFillerTest
Program: ArionTableFillerTest {type=VOID, tab_index=37, lev=0}
|
|-- Declarations {type=VOID, tab_index=-1, lev=0}
| |
| | |-- TypeDeclaration: BarisData {type=ARRAY, tab_index=40, lev=0}
| | | |
| | | |-- ArrayType {type=ARRAY, tab_index=-1, lev=0}
| | | |
| | | |-- RangeType {type=VOID, tab_index=-1, lev=0}
| | | |
| | | |-- Literal: 1 {type=VOID, tab_index=-1, lev=0}
```

```

| | | | |
| | | | | \-- Literal: 3 {type=VOID, tab_index=-1, lev=0}
| | | | |
| | | | | \-- PrimitiveType: integer {type=VOID, tab_index=-1, lev=0}
| | | | |
| | | | |
| | | | | +-- TypeDeclaration: MatriksData {type=ARRAY, tab_index=41, lev=0}
| | | | |
| | | | | | |
| | | | | | | \-- ArrayType {type=ARRAY, tab_index=-1, lev=0}
| | | | | | |
| | | | | | | +-- RangeType {type=VOID, tab_index=-1, lev=0}
| | | | | | |
| | | | | | | | |
| | | | | | | | | +-- Literal: 1 {type=VOID, tab_index=-1, lev=0}
| | | | | | | | |
| | | | | | | | | \-- Literal: 4 {type=VOID, tab_index=-1, lev=0}
| | | | | | | | |
| | | | | | | | | \-- NamedType: BarisData {type=VOID, tab_index=-1, lev=0}
| | | | | | | | |
| | | | | | | | | +-- VarDecl(name: 'myMatrix', type: 'MatriksData') {type=ARRAY, tab_index=42, lev=0}
| | | | | | | | |
| | | | | | | | | +-- VarDecl(name: 'globalCounter', type: 'integer') {type=INTEGER, tab_index=43, lev=0}
| | | | | | | | |
| | | | | | | | | +-- SubprogramDeclaration: HitungKalkulasiKompleks {type=VOID, tab_index=38, lev=0}
| | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | | +-- PrimitiveType: integer {type=INTEGER, tab_index=-1, lev=0}
| | | | | | | | | | |
| | | | | | | | | | | +-- PrimitiveType: integer {type=INTEGER, tab_index=-1, lev=0}
| | | | | | | | | | |
| | | | | | | | | | | +-- PrimitiveType: boolean {type=BOOLEAN, tab_index=-1, lev=0}
| | | | | | | | | | |
| | | | | | | | | | | +-- PrimitiveType: char {type=CHAR, tab_index=-1, lev=0}
| | | | | | | | | | |
| | | | | | | | | | | +-- VarDecl(name: 'lokalX', type: 'integer') {type=INTEGER, tab_index=48, lev=1}
| | | | | | | | | | |
| | | | | | | | | | | \-- Block {type=VOID, tab_index=-1, lev=0}
| | | | | | | | | | |
| | | | | | | | | | | | |
| | | | | | | | | | | | | +-- Assignment {type=VOID, tab_index=-1, lev=0}
| | | | | | | | | | | | |
| | | | | | | | | | | | | +-- Variable: lokalX {type=INTEGER, tab_index=48, lev=1}
| | | | | | | | | | | | |
| | | | | | | | | | | | | \-- BinaryOp: + {type=INTEGER, tab_index=-1, lev=0}
| | | | | | | | | | | | |
| | | | | | | | | | | | | +-- Variable: p1 {type=INTEGER, tab_index=44, lev=1}
| | | | | | | | | | | | |
| | | | | | | | | | | | | \-- Variable: p2 {type=INTEGER, tab_index=45, lev=1}
| | | | | | | | | | | | |
| | | | | | | | | | | | | +-- If {type=VOID, tab_index=-1, lev=1}
| | | | | | | | | | | | |
| | | | | | | | | | | | | +-- Variable: opsi {type=BOOLEAN, tab_index=46, lev=1}
| | | | | | | | | | | | |
| | | | | | | | | | | | | +-- CallStatement {type=VOID, tab_index=-1, lev=0}

```

```

|-- Call: writeln {type=VOID, tab_index=33, lev=0}
|
|-- Variable: flag {type=CHAR, tab_index=47, lev=1}
|
|-- CallStatement {type=VOID, tab_index=-1, lev=0}
|
|-- Call: writeln {type=VOID, tab_index=33, lev=0}
|
|-- Literal: Selesai {type=STRING, tab_index=-1, lev=0}
|
|-- EmptyStatement {type=VOID, tab_index=-1, lev=0}
|
|-- FunctionDeclaration: PengecekanMatriks(m; batas) : boolean {type=BOOLEAN,
tab_index=39, lev=0}
|
|+-- NamedType: MatriksData {type=ARRAY, tab_index=-1, lev=0}
|
|+-- PrimitiveType: integer {type=INTEGER, tab_index=-1, lev=0}
|
|-- Block {type=VOID, tab_index=-1, lev=0}
|
|+-- If {type=VOID, tab_index=-1, lev=1}
|
|+-- BinaryOp: > {type=BOOLEAN, tab_index=-1, lev=0}
|
|+-- Variable: m[][] {type=INTEGER, tab_index=49, lev=1}
|
|+-- Literal: 1 {type=INTEGER, tab_index=-1, lev=0}
|
|+-- Literal: 1 {type=INTEGER, tab_index=-1, lev=0}
|
|+-- Variable: batas {type=INTEGER, tab_index=50, lev=1}
|
|+-- Assignment {type=VOID, tab_index=-1, lev=0}
|
|+-- Variable: PengecekanMatriks {type=BOOLEAN, tab_index=39, lev=0}
|
|+-- Variable: true {type=BOOLEAN, tab_index=35, lev=0}
|
|-- Assignment {type=VOID, tab_index=-1, lev=0}
|
|+-- Variable: PengecekanMatriks {type=BOOLEAN, tab_index=39, lev=0}
|
|+-- Variable: false {type=BOOLEAN, tab_index=36, lev=0}
|
|-- EmptyStatement {type=VOID, tab_index=-1, lev=0}
|
|-- Block {type=VOID, tab_index=-1, lev=0}
|

```

```

+-- Assignment {type=VOID, tab_index=-1, lev=0}
|
|   +-- Variable: globalCounter {type=INTEGER, tab_index=43, lev=0}
|   |
|   \-- Literal: 10 {type=INTEGER, tab_index=-1, lev=0}
|
+-- Assignment {type=VOID, tab_index=-1, lev=0}
|
|   +-- Variable: myMatrix[][] {type=INTEGER, tab_index=42, lev=0}
|   |
|   |   +-- Literal: 1 {type=INTEGER, tab_index=-1, lev=0}
|   |   |
|   |   \-- Literal: 1 {type=INTEGER, tab_index=-1, lev=0}
|   |
|   \-- Literal: 5 {type=INTEGER, tab_index=-1, lev=0}
|
+-- CallStatement {type=VOID, tab_index=-1, lev=0}
|
|   \-- Call: HitungKalkulasiKompleks {type=VOID, tab_index=38, lev=0}
|   |
|   |   +-- Literal: 5 {type=INTEGER, tab_index=-1, lev=0}
|   |   |
|   |   +-- Literal: 10 {type=INTEGER, tab_index=-1, lev=0}
|   |   |
|   |   +-- Variable: true {type=BOOLEAN, tab_index=35, lev=0}
|   |   |
|   |   \-- Literal: A {type=CHAR, tab_index=-1, lev=0}
|   |
|   +-- If {type=VOID, tab_index=-1, lev=0}
|   |
|   |   +-- Call: PengecekanMatriks {type=BOOLEAN, tab_index=39, lev=0}
|   |   |
|   |   |   +-- Variable: myMatrix {type=ARRAY, tab_index=42, lev=0}
|   |   |   |
|   |   |   \-- Variable: globalCounter {type=INTEGER, tab_index=43, lev=0}
|   |   |
|   |   +-- CallStatement {type=VOID, tab_index=-1, lev=0}
|   |   |
|   |   |   \-- Call: writeln {type=VOID, tab_index=33, lev=0}
|   |   |   |
|   |   |   |   \-- Literal: Nilai matriks aman. {type=STRING, tab_index=-1, lev=0}
|   |   |
|   |   \-- CallStatement {type=VOID, tab_index=-1, lev=0}
|   |   |
|   |   |   \-- Call: writeln {type=VOID, tab_index=33, lev=0}
|   |   |   |
|   |   |   |   \-- Literal: Nilai matriks di bawah batas. {type=STRING, tab_index=-1, lev=0}
|   |
|   +-- EmptyStatement {type=VOID, tab_index=-1, lev=0}

```

=== Symbol Tables ===

tab :

idx	id	obj	type	ref	nrm	lev	adr	link
0	<dummy>		keyword	void	0	1	0	0
1	and		keyword	void	0	1	0	4
2	array		keyword	void	0	1	0	8
3	begin		keyword	void	0	1	0	12
4	case		keyword	void	0	1	0	16
5	const		keyword	void	0	1	0	20
6	div		keyword	void	0	1	0	24
7	downto		keyword	void	0	1	0	28
8	do		keyword	void	0	1	0	32
9	else		keyword	void	0	1	0	36
10	end		keyword	void	0	1	0	40
11	for		keyword	void	0	1	0	44
12	function		keyword	void	0	1	0	48
13	if		keyword	void	0	1	0	52
14	mod		keyword	void	0	1	0	56
15	not		keyword	void	0	1	0	60
16	of		keyword	void	0	1	0	64
17	or		keyword	void	0	1	0	68
18	procedure		keyword	void	0	1	0	72
19	program		keyword	void	0	1	0	76
20	record		keyword	void	0	1	0	80
21	repeat		keyword	void	0	1	0	84
22	then		keyword	void	0	1	0	88
23	to		keyword	void	0	1	0	92
24	type		keyword	void	0	1	0	96
25	until		keyword	void	0	1	0	100
26	var		keyword	void	0	1	0	104
27	while		keyword	void	0	1	0	108
28	integer		type	integer	0	1	0	112
29	real		type	real	0	1	0	116
30	char		type	char	0	1	0	120
31	boolean		type	boolean	0	1	0	124
32	string		type	string	0	1	0	128
33	writeln		procedure	void	0	1	0	132
34	println		procedure	void	0	1	0	136
35	true		constant	boolean	0	1	0	140
36	false		constant	boolean	0	1	0	144
37	ArionTableFillerTest		program	void	0	1	0	148
38	HitungKalkulasiKompleks		procedure	void	1	1	0	152
39	PengecekanMatriks		function	boolean	2	1	0	156
40	BarisData		type	array	0	1	0	160
41	MatriksData		type	array	1	1	0	172
42	myMatrix		variabel	array	1	1	0	220
43	globalCounter		variabel	integer	0	1	0	268
44	p1		parameter	integer	0	1	1	0
45	p2		parameter	integer	0	1	1	4

```

46 opsi          parameter boolean 0 1 1 8 45
47 flag          parameter char 0 1 1 9 46
48 lokalX        variabel integer 0 1 1 10 47
49 m             parameter array 0 1 1 0 0
50 batas         parameter integer 0 1 1 48 49

```

```

btab :
idx last lparam psize vsize

```

```

-----
0 43 0 0 0
1 48 47 10 14
2 50 50 52 52

```

```

atab :
idx inxtyp eltyp ref low high elsz size

```

```

-----
0 range integer 0 1 3 4 12
1 range array 0 1 4 12 48

```

Semantic report berhasil disimpan di: ../test/milestone-3/output/TestIsiBtabAtab-ast.txt

=== Semantic Warnings ===

[line 35, col 1] Warning: Statement ini tidak akan pernah dijalankan karena statement sebelumnya sudah mengembalikan nilai function.

=====

Selesai.

Pengujian 9

1. Nama Pengujian : Program Sederhana
2. Input file : Capek.txt
3. Output file : Capek-ast.txt
4. Deskripsi : Pengujian program sederhana Arion
5. Hasil Pengujian :

Tabel 4.9 Pengujian 9

Input

program ArionUltimateEnterpriseTest;


```

const
    MaksGrup == 3;
    MaksStaf == 2;
    BonusAdmin == 1500;

type
    { * 1. Registrasi Dua Tipe Enum *}
    DivisiKaryawan == (IT, Keuangan, HRD);
    LevelPangkat == (Junior, Senior, Manager);

    { * 2. Record Tingkat Dalam (Sistem Kontrak) *}
    SkemaKontrak == record
        pangkat : LevelPangkat;
        isAktif : boolean;
        masaKerja : integer
    end;

    { * 3. Record Tingkat Menengah (Detail Profil) Bersarang *}
    ProfilKaryawan == record
        idKaryawan : integer;
        divisi : DivisiKaryawan;
        kontrak : SkemaKontrak { * Nested Record *}
    end;

    { * 4. Array Multi-Dimensi dari Record *}
    { * Struktur: Array [Grup] -> Array [Staf] -> Struktur Record Profil *}
    MatriksPerusahaan == array[1..3] of array[1..2] of ProfilKaryawan;

var
    korporasi : MatriksPerusahaan;
    grupIdx : integer;
    stafIdx : integer;
    totalGaji : integer;

begin
    { * --- TAHAP 1: EKSTREM ASSIGNMENT & EVALUASI ALAMAT (ADR) --- *}
    grupIdx := 2;
    stafIdx := 1;

    { * Akses Jalur Sutra: Array 2D -> Record -> Record -> Field *}
    korporasi[grupIdx][stafIdx].idKaryawan := 777;
    korporasi[grupIdx][stafIdx].divisi := IT;

```

```

    korporasi[grupIdx][staflIdx].kontrak.pangkat := Senior;
    korporasi[grupIdx][staflIdx].kontrak.isAktif := true;
    korporasi[grupIdx][staflIdx].kontrak.masaKerja := 5;

    totalGaji := 5000;

    {* --- TAHAP 2: PENGUJIAN INTEGRASI NESTED CASE STATEMENT --- *}
    {* Menguji Parser & Semantik terhadap scope Case bertingkat tanpa Else *}
    case korporasi[grupIdx][staflIdx].divisi of
        IT:
            {* Case Tingkat 2: Mengecek pangkat di dalam nested record *}
            case korporasi[grupIdx][staflIdx].kontrak.pangkat of
                Junior:
                    totalGaji := totalGaji + 500;
                Senior:
                    totalGaji := totalGaji + 2000;
                Manager:
                    totalGaji := totalGaji + 5000
            end;
        Keuangan, HRD:
            totalGaji := totalGaji + BonusAdmin;
    end;

    {* --- TAHAP 3: OUTPUT AKHIR --- *}
    writeln('Proses Semantik Selesai. Total Gaji Terhitung:');
    writeln(totalGaji);
end.

```

Output

```

Processing Semantic Analysis...
Menganalisis Program: ArionUltimateEnterpriseTest
Program: ArionUltimateEnterpriseTest {type=VOID, tab_index=37, lev=0}
|
+-- Declarations {type=VOID, tab_index=-1, lev=0}
| |
| +-- ConstDeclaration: MaksGrup {type=INTEGER, tab_index=38, lev=0}
| | |
| | \-- Literal: 3 {type=INTEGER, tab_index=-1, lev=0}
| |
| +-- ConstDeclaration: MaksStaf {type=INTEGER, tab_index=39, lev=0}

```

```

| | |
| | | \-- Literal: 2 {type=INTEGER, tab_index=-1, lev=0}
| | |
| | | +--- ConstDeclaration: BonusAdmin {type=INTEGER, tab_index=40, lev=0}
| | |
| | | \-- Literal: 1500 {type=INTEGER, tab_index=-1, lev=0}
| | |
| | | +--- TypeDeclaration: DivisiKaryawan {type=ENUMERATED, tab_index=44, lev=0}
| | |
| | | \-- EnumeratedType {type=ENUMERATED, tab_index=-1, lev=0}
| | |
| | | +--- TypeDeclaration: LevelPangkat {type=ENUMERATED, tab_index=48, lev=0}
| | |
| | | \-- EnumeratedType {type=ENUMERATED, tab_index=-1, lev=0}
| | |
| | | +--- TypeDeclaration: SkemaKontrak {type=RECORD, tab_index=49, lev=0}
| | |
| | | \-- RecordType(pangkat; isAktif; masaKerja) {type=RECORD, tab_index=-1,
lev=0}
| | |
| | | |
| | | | +--- NamedType: LevelPangkat {type=ENUMERATED, tab_index=-1, lev=0}
| | | |
| | | | +--- PrimitiveType: boolean {type=BOOLEAN, tab_index=-1, lev=0}
| | | |
| | | | \-- PrimitiveType: integer {type=INTEGER, tab_index=-1, lev=0}
| | | |
| | | | +--- TypeDeclaration: ProfilKaryawan {type=RECORD, tab_index=53, lev=0}
| | | |
| | | | \-- RecordType(idKaryawan; divisi; kontrak) {type=RECORD, tab_index=-1,
lev=0}
| | | |
| | | | |
| | | | | +--- PrimitiveType: integer {type=INTEGER, tab_index=-1, lev=0}
| | | | |
| | | | | +--- NamedType: DivisiKaryawan {type=ENUMERATED, tab_index=-1, lev=0}
| | | | |
| | | | | \-- NamedType: SkemaKontrak {type=RECORD, tab_index=-1, lev=0}
| | | | |
| | | | | +--- TypeDeclaration: MatriksPerusahaan {type=ARRAY, tab_index=57, lev=0}
| | | | |
| | | | | \-- ArrayType {type=ARRAY, tab_index=-1, lev=0}
| | | | |
| | | | | +--- RangeType {type=VOID, tab_index=-1, lev=0}

```

```

| | | |
| | | | +--- Literal: 1 {type=VOID, tab_index=-1, lev=0}
| | | | |
| | | | \-- Literal: 3 {type=VOID, tab_index=-1, lev=0}
| | | | |
| | | | \-- ArrayType {type=VOID, tab_index=-1, lev=0}
| | | | |
| | | | +--- RangeType {type=VOID, tab_index=-1, lev=0}
| | | | |
| | | | | +--- Literal: 1 {type=VOID, tab_index=-1, lev=0}
| | | | | |
| | | | | \-- Literal: 2 {type=VOID, tab_index=-1, lev=0}
| | | | | |
| | | | | \-- NamedType: ProfilKaryawan {type=VOID, tab_index=-1, lev=0}
| | | | |
| | | | +--- VarDecl(name: 'korporasi', type: 'MatriksPerusahaan') {type=ARRAY,
| | | | tab_index=58, lev=0}
| | | | |
| | | | +--- VarDecl(name: 'grupIdx', type: 'integer') {type=INTEGER, tab_index=59,
| | | | lev=0}
| | | | |
| | | | +--- VarDecl(name: 'stafIdx', type: 'integer') {type=INTEGER, tab_index=60, lev=0}
| | | | |
| | | | \-- VarDecl(name: 'totalGaji', type: 'integer') {type=INTEGER, tab_index=61,
| | | | lev=0}
| | | | |
| | | | \-- Block {type=VOID, tab_index=-1, lev=0}
| | | | |
| | | | | +--- Assignment {type=VOID, tab_index=-1, lev=0}
| | | | | |
| | | | | | +--- Variable: grupIdx {type=INTEGER, tab_index=59, lev=0}
| | | | | | |
| | | | | | \-- Literal: 2 {type=INTEGER, tab_index=-1, lev=0}
| | | | | | |
| | | | | +--- Assignment {type=VOID, tab_index=-1, lev=0}
| | | | | |
| | | | | | +--- Variable: stafIdx {type=INTEGER, tab_index=60, lev=0}
| | | | | | |
| | | | | | \-- Literal: 1 {type=INTEGER, tab_index=-1, lev=0}
| | | | | | |
| | | | | +--- Assignment {type=VOID, tab_index=-1, lev=0}
| | | | | |
| | | | |

```

```

|   +-- Variable: korporasi[idKaryawan {type=INTEGER, tab_index=58, lev=0}
|   |   |
|   |   +-- Variable: grupIdx {type=INTEGER, tab_index=59, lev=0}
|   |   |
|   |   \-- Variable: stafIdx {type=INTEGER, tab_index=60, lev=0}
|   |
|   \-- Literal: 777 {type=INTEGER, tab_index=-1, lev=0}
|
+-- Assignment {type=VOID, tab_index=-1, lev=0}
|   |
|   +-- Variable: korporasi[divisi {type=ENUMERATED, tab_index=58, lev=0}
|   |   |
|   |   +-- Variable: grupIdx {type=INTEGER, tab_index=59, lev=0}
|   |   |
|   |   \-- Variable: stafIdx {type=INTEGER, tab_index=60, lev=0}
|   |
|   \-- Variable: IT {type=ENUMERATED, tab_index=41, lev=0}
|
+-- Assignment {type=VOID, tab_index=-1, lev=0}
|   |
|   +-- Variable: korporasi[kontrak.pangkat {type=ENUMERATED, tab_index=58,
lev=0}
|   |   |
|   |   +-- Variable: grupIdx {type=INTEGER, tab_index=59, lev=0}
|   |   |
|   |   \-- Variable: stafIdx {type=INTEGER, tab_index=60, lev=0}
|   |
|   \-- Variable: Senior {type=ENUMERATED, tab_index=46, lev=0}
|
+-- Assignment {type=VOID, tab_index=-1, lev=0}
|   |
|   +-- Variable: korporasi[kontrak.isAktif {type=BOOLEAN, tab_index=58,
lev=0}
|   |   |
|   |   +-- Variable: grupIdx {type=INTEGER, tab_index=59, lev=0}
|   |   |
|   |   \-- Variable: stafIdx {type=INTEGER, tab_index=60, lev=0}
|   |
|   \-- Variable: true {type=BOOLEAN, tab_index=35, lev=0}
|
+-- Assignment {type=VOID, tab_index=-1, lev=0}
|   |

```

```

| +-- Variable: korporasi[][].kontrak.masaKerja {type=INTEGER, tab_index=58,
lev=0}
| | |
| | | +-- Variable: grupIdx {type=INTEGER, tab_index=59, lev=0}
| | |
| | | \-- Variable: stafIdx {type=INTEGER, tab_index=60, lev=0}
| | |
| | | \-- Literal: 5 {type=INTEGER, tab_index=-1, lev=0}
| | |
| +-- Assignment {type=VOID, tab_index=-1, lev=0}
| |
| | +-- Variable: totalGaji {type=INTEGER, tab_index=61, lev=0}
| | |
| | | \-- Literal: 5000 {type=INTEGER, tab_index=-1, lev=0}
| | |
| +-- Case {type=VOID, tab_index=-1, lev=0}
| |
| | +-- Variable: korporasi[][].divisi {type=ENUMERATED, tab_index=58, lev=0}
| | |
| | | +-- Variable: grupIdx {type=INTEGER, tab_index=59, lev=0}
| | | |
| | | | \-- Variable: stafIdx {type=INTEGER, tab_index=60, lev=0}
| | | |
| | | +-- Variable: IT {type=ENUMERATED, tab_index=41, lev=0}
| | | |
| | | +-- Case {type=VOID, tab_index=-1, lev=0}
| | | |
| | | | +-- Variable: korporasi[][].kontrak.pangkat {type=ENUMERATED,
tab_index=58, lev=0}
| | | | |
| | | | | +-- Variable: grupIdx {type=INTEGER, tab_index=59, lev=0}
| | | | |
| | | | | \-- Variable: stafIdx {type=INTEGER, tab_index=60, lev=0}
| | | | |
| | | | +-- Variable: Junior {type=ENUMERATED, tab_index=45, lev=0}
| | | | |
| | | | +-- Assignment {type=VOID, tab_index=-1, lev=0}
| | | | |
| | | | | +-- Variable: totalGaji {type=INTEGER, tab_index=61, lev=0}
| | | | |
| | | | | \-- BinaryOp: + {type=INTEGER, tab_index=-1, lev=0}
| | | |

```

```

| | |   +-- Variable: totalGaji {type=INTEGER, tab_index=61, lev=0}
| | |   |
| | |   \-- Literal: 500 {type=INTEGER, tab_index=-1, lev=0}
| | |
| | | +-- Variable: Senior {type=ENUMERATED, tab_index=46, lev=0}
| | | |
| | | +-- Assignment {type=VOID, tab_index=-1, lev=0}
| | | |
| | | |   +-- Variable: totalGaji {type=INTEGER, tab_index=61, lev=0}
| | | |   |
| | | |   \-- BinaryOp: + {type=INTEGER, tab_index=-1, lev=0}
| | | |   |
| | | |   |   +-- Variable: totalGaji {type=INTEGER, tab_index=61, lev=0}
| | | |   |   |
| | | |   |   \-- Literal: 2000 {type=INTEGER, tab_index=-1, lev=0}
| | | |
| | | +-- Variable: Manager {type=ENUMERATED, tab_index=47, lev=0}
| | | |
| | | \-- Assignment {type=VOID, tab_index=-1, lev=0}
| | | |
| | | |   +-- Variable: totalGaji {type=INTEGER, tab_index=61, lev=0}
| | | |   |
| | | |   \-- BinaryOp: + {type=INTEGER, tab_index=-1, lev=0}
| | | |   |
| | | |   |   +-- Variable: totalGaji {type=INTEGER, tab_index=61, lev=0}
| | | |   |   |
| | | |   |   \-- Literal: 5000 {type=INTEGER, tab_index=-1, lev=0}
| | | |
| | | +-- Variable: Keuangan {type=ENUMERATED, tab_index=42, lev=0}
| | | |
| | | +-- Variable: HRD {type=ENUMERATED, tab_index=43, lev=0}
| | | |
| | | \-- Assignment {type=VOID, tab_index=-1, lev=0}
| | | |
| | | |   +-- Variable: totalGaji {type=INTEGER, tab_index=61, lev=0}
| | | |   |
| | | |   \-- BinaryOp: + {type=INTEGER, tab_index=-1, lev=0}
| | | |   |
| | | |   |   +-- Variable: totalGaji {type=INTEGER, tab_index=61, lev=0}
| | | |   |   |
| | | |   |   \-- Variable: BonusAdmin {type=INTEGER, tab_index=40, lev=0}

```

```

+-- CallStatement {type=VOID, tab_index=-1, lev=0}
| |
| | \-- Call: writeln {type=VOID, tab_index=33, lev=0}
| | |
| | | \-- Literal: Proses Semantik Selesai. Total Gaji Terhitung: {type=STRING,
tab_index=-1, lev=0}
|
+-- CallStatement {type=VOID, tab_index=-1, lev=0}
| |
| | \-- Call: writeln {type=VOID, tab_index=33, lev=0}
| | |
| | | \-- Variable: totalGaji {type=INTEGER, tab_index=61, lev=0}
|
\-- EmptyStatement {type=VOID, tab_index=-1, lev=0}

```

=== Symbol Tables ===

tab :

idx	id	obj	type	ref	nrm	lev	adr	link
0	<dummy>	keyword	void	0	1	0	0	0
1	and	keyword	void	0	1	0	4	0
2	array	keyword	void	0	1	0	8	1
3	begin	keyword	void	0	1	0	12	2
4	case	keyword	void	0	1	0	16	3
5	const	keyword	void	0	1	0	20	4
6	div	keyword	void	0	1	0	24	5
7	downto	keyword	void	0	1	0	28	6
8	do	keyword	void	0	1	0	32	7
9	else	keyword	void	0	1	0	36	8
10	end	keyword	void	0	1	0	40	9
11	for	keyword	void	0	1	0	44	10
12	function	keyword	void	0	1	0	48	11
13	if	keyword	void	0	1	0	52	12
14	mod	keyword	void	0	1	0	56	13
15	not	keyword	void	0	1	0	60	14
16	of	keyword	void	0	1	0	64	15
17	or	keyword	void	0	1	0	68	16
18	procedure	keyword	void	0	1	0	72	17
19	program	keyword	void	0	1	0	76	18
20	record	keyword	void	0	1	0	80	19
21	repeat	keyword	void	0	1	0	84	20
22	then	keyword	void	0	1	0	88	21

23	to	keyword	void	0	1	0	92	22
24	type	keyword	void	0	1	0	96	23
25	until	keyword	void	0	1	0	100	24
26	var	keyword	void	0	1	0	104	25
27	while	keyword	void	0	1	0	108	26
28	integer	type	integer	0	1	0	112	27
29	real	type	real	0	1	0	116	28
30	char	type	char	0	1	0	120	29
31	boolean	type	boolean	0	1	0	124	30
32	string	type	string	0	1	0	128	31
33	writeln	procedure	void	0	1	0	132	32
34	println	procedure	void	0	1	0	136	33
35	true	constant	boolean	0	1	0	140	34
36	false	constant	boolean	0	1	0	144	35
37	ArionUltimateEnterpriseTest	program	void	0	1	0	148	36
38	MaksGrup	constant	integer	0	1	0	152	37
39	MaksStaf	constant	integer	0	1	0	156	38
40	BonusAdmin	constant	integer	0	1	0	160	39
41	IT	constant	enumerated	0	1	0	0	40
42	Keuangan	constant	enumerated	0	1	0	1	41
43	HRD	constant	enumerated	0	1	0	2	42
44	DivisiKaryawan	type	enumerated	0	1	0	176	43
45	Junior	constant	enumerated	0	1	0	0	44
46	Senior	constant	enumerated	0	1	0	1	45
47	Manager	constant	enumerated	0	1	0	2	46
48	LevelPangkat	type	enumerated	0	1	0	192	47
49	SkemaKontrak	type	record	1	1	0	196	48
50	pangkat	field	enumerated	0	1	1	0	0
51	isAktif	field	boolean	0	1	1	4	50
52	masaKerja	field	integer	0	1	1	5	51
53	ProfilKaryawan	type	record	2	1	0	205	49
54	idKaryawan	field	integer	0	1	1	0	0
55	divisi	field	enumerated	0	1	1	4	54
56	kontrak	field	record	1	1	1	8	55
57	MatriksPerusahaan	type	array	1	1	0	222	53
58	korporasi	variabel	array	1	1	0	324	57
59	grupIdx	variabel	integer	0	1	0	426	58
60	stafIdx	variabel	integer	0	1	0	430	59
61	totalGaji	variabel	integer	0	1	0	434	60

btab :

idx last lparam psize vsize

```
-----
0 61 0 0 0
1 52 0 0 9
2 56 0 0 17
```

atab :

```
idx inxtyp eltyp ref low high elsz size
```

```
-----
0 range record 53 1 2 17 34
1 range array 0 1 3 34 102
```

Semantic report berhasil disimpan di: ../test/milestone-3/output/Capek-ast.txt

Selesai.

Pengujian 10

1. Nama Pengujian : Nesting Loop
2. Input file : NestingJahat.txt
3. Output file : NestingJahat-ast.txt
4. Deskripsi : Menguji nesting yang banyak pada for maupun while serta repeat
5. Hasil Pengujian :

Tabel 4.10 Pengujian 10

Input

```
program DeepNest;

var
  a, b, c, d : integer;

begin
  for a := 1 to 2 do
    begin
      for b := 1 to 2 do
        begin
          while b < 3 do
            begin
```

```

repeat
  for c := 1 to 2 do
    begin
      for d := 1 to 2 do
        begin
          writeln(a, b, c, d);
        end;
      end;
    end;

    b := b + 1;
  until b >= 3;
end;
end;
end.

```

Output

Processing Semantic Analysis...

Menganalisis Program: DeepNest

Program: DeepNest {type=VOID, tab_index=37, lev=0}

```

|
+-- Declarations {type=VOID, tab_index=-1, lev=0}
| |
| | +-- VarDecl(name: 'a', type: 'integer') {type=INTEGER, tab_index=38, lev=0}
| | |
| | +-- VarDecl(name: 'b', type: 'integer') {type=INTEGER, tab_index=39, lev=0}
| | |
| | +-- VarDecl(name: 'c', type: 'integer') {type=INTEGER, tab_index=40, lev=0}
| | |
| | \-- VarDecl(name: 'd', type: 'integer') {type=INTEGER, tab_index=41, lev=0}
| |
| \-- Block {type=VOID, tab_index=-1, lev=0}
| |
| | +-- For: a {type=VOID, tab_index=38, lev=0}
| | |
| | | +-- Literal: 1 {type=INTEGER, tab_index=-1, lev=0}
| | | |
| | | +-- Literal: 2 {type=INTEGER, tab_index=-1, lev=0}
| | | |
| | | \-- Block {type=VOID, tab_index=-1, lev=0}

```


tab :

idx id obj type ref nrm lev adr link

0	<dummy>	keyword	void	0	1	0	0	0
1	and	keyword	void	0	1	0	4	0
2	array	keyword	void	0	1	0	8	1
3	begin	keyword	void	0	1	0	12	2
4	case	keyword	void	0	1	0	16	3
5	const	keyword	void	0	1	0	20	4
6	div	keyword	void	0	1	0	24	5
7	downto	keyword	void	0	1	0	28	6
8	do	keyword	void	0	1	0	32	7
9	else	keyword	void	0	1	0	36	8
10	end	keyword	void	0	1	0	40	9
11	for	keyword	void	0	1	0	44	10
12	function	keyword	void	0	1	0	48	11
13	if	keyword	void	0	1	0	52	12
14	mod	keyword	void	0	1	0	56	13
15	not	keyword	void	0	1	0	60	14
16	of	keyword	void	0	1	0	64	15
17	or	keyword	void	0	1	0	68	16
18	procedure	keyword	void	0	1	0	72	17
19	program	keyword	void	0	1	0	76	18
20	record	keyword	void	0	1	0	80	19
21	repeat	keyword	void	0	1	0	84	20
22	then	keyword	void	0	1	0	88	21
23	to	keyword	void	0	1	0	92	22
24	type	keyword	void	0	1	0	96	23
25	until	keyword	void	0	1	0	100	24
26	var	keyword	void	0	1	0	104	25
27	while	keyword	void	0	1	0	108	26
28	integer	type	integer	0	1	0	112	27
29	real	type	real	0	1	0	116	28
30	char	type	char	0	1	0	120	29
31	boolean	type	boolean	0	1	0	124	30
32	string	type	string	0	1	0	128	31
33	writeln	procedure	void	0	1	0	132	32
34	println	procedure	void	0	1	0	136	33
35	true	constant	boolean	0	1	0	140	34
36	false	constant	boolean	0	1	0	144	35
37	DeepNest	program	void	0	1	0	148	36
38	a	variabel	integer	0	1	0	152	37

```
39 b      variabel integer 0 1 0 156 38
40 c      variabel integer 0 1 0 160 39
41 d      variabel integer 0 1 0 164 40
```

btab :

idx last lparam psize vsize

0 41 0 0 0

atab :

idx inxtyp eltyp ref low high elsz size

Semantic report berhasil disimpan di: ../test/milestone-3/output/NestingJahat-ast.txt

Selesai.

Bab 5

Kesimpulan dan Saran

5.1. Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa:

- Implementasi *Semantic Analyzer* berhasil dibangun secara modular dan terstruktur menggunakan arsitektur *Visitor Pattern* dalam menelusuri *Abstract Syntax Tree* (AST).
- Sistem manajemen memori dan resolusi lingkup (*scope*) beroperasi dengan akurat melalui integrasi tiga tabel utama, yaitu *Identifier Table*, *Array Table*, dan *Block Table*.
- Kompilator telah terbukti tangguh dalam memblokir pelanggaran semantik, termasuk modifikasi konstanta ilegal, ketidaksesuaian aritas parameter, dan konflik tipe data.

5.2. Saran

Beberapa hal yang dapat dikembangkan lebih lanjut untuk meningkatkan kualitas program pada pengembangan selanjutnya adalah:

- Evaluasi batas indeks array (*array bound checking*) sebaiknya diimplementasikan sebagai instruksi *runtime* pada tahap *Code Generation* agar proses kompilasi tetap efisien.
- Atribut kedalaman leksikal (*level*) dan offset memori (*address*) pada Tabel Simbol sudah sangat stabil, sehingga direkomendasikan untuk langsung dipetakan ke dalam struktur *Stack Frame* pada *Milestone* selanjutnya.
- Sistem pelaporan dapat ditingkatkan dengan menambahkan mekanisme *Error Recovery* dasar, sehingga kompilator dapat menemukan lebih banyak kesalahan semantik dalam satu kali siklus eksekusi sebelum benar-benar dihentikan.

Lampiran

Tautan Repositori Github:

<https://github.com/Irvin-Tandiarrang-Sumual/STI-Tubes-IF2224-2026.git>

NIM	Nama	Deskripsi Tugas	Persentase Kontribusi
13524008	Agatha Tatianingseto	Kerja	20%
13524030	Irvin Tandiarrang Sumual	Kerja	20%
13524074	Bernhard Aprillio Pramana	Kerja	20%
13524096	Moreno Syawali Ganda Sugita	Kerja	20%
13524110	Jennifer Khang	Kerja	20%

Daftar Referensi

GeeksforGeeks. (2024). *Semantic Analysis in Compiler Design*. [Online]. Available:

<https://www.geeksforgeeks.org/compiler-design/semantic-analysis-in-compiler-design/>.

Accessed 08 May 2026.

TutorialsPoint. (n.d.). *Compiler Design - Semantic Analysis*. [Online]. Available:

https://www.tutorialspoint.com/compiler_design/compiler_design_semantic_analysis.htm.

Accessed 08 May 2026.

GeeksforGeeks. (2024). *Abstract Syntax Tree vs Parse Tree*. [Online]. Available:

<https://www.geeksforgeeks.org/compiler-design/abstract-syntax-tree-vs-parse-tree/>.

Accessed 08 May 2026.

GeeksforGeeks. (2024). *Symbol Table in Compiler*. [Online]. Available:

<https://www.geeksforgeeks.org/compiler-design/symbol-table-compiler/>. Accessed 08

May 2026.

TutorialsPoint. (n.d.). *Compiler Design - Symbol Table*. [Online]. Available:

https://www.tutorialspoint.com/compiler_design/compiler_design_symbol_table.htm.

Accessed 08 May 2026.